Build from source (CMake)

SWAT+ is built out-of-source. All intermediate and final files go into a separate build directory so the source tree stays clean. The same source tree can host multiple build flavors (different compilers, debug vs release) at the same time.

The build is driven by `CMakeLists.txt` at the repository root. It globs every `.f90` under `src/` and links a single executable.

```
file(GLOB sources src/*.f90)
add_executable(${SWATPLUS_EXE} ${sources})
```

Generate the build files

Make `swatplus` your current working directory.

```
cd <dir>/swatplus
```

Configure the build. This processes `CMakeLists.txt` and writes a build system into `build/`. It does not compile yet.

```
cmake -S . -B build
```

The short form is equivalent.

```
cmake -B build
```

Pick a compiler

Default is `gfortran`. To force it explicitly:

```
cmake -B build -D CMAKE_Fortran_COMPILER=gfortran
```

For Intel Fortran, `ifort` was deprecated in 2024 and removed from the 2025 Intel oneAPI release. Use `ifx` instead.

```
cmake -B build -D CMAKE_Fortran_COMPILER=ifx
```

On Windows, the Intel compilers need a generator toolset:

```
cmake -B build -D CMAKE_GENERATOR_TOOLSET=fortran=ifx
cmake -B build -G "Ninja" -D CMAKE_Fortran_COMPILER=gfortran
```

Note: on Windows, use the default `cmd` shell, not PowerShell, for the command-line build.

Debug vs release

For a debug build with symbols:

```
cmake -B build -D CMAKE_BUILD_TYPE=Debug
```

For a release build:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
```

Compile and link

From the `swatplus` directory:

```
cmake --build build
```

Parallelize across CPU cores with `-j`. Eight is a reasonable default on most laptops.

```
cmake --build build -j 8
```

The executable lands in `build/` as `swatplus-<version>-<arch>.exe` on Windows and without an extension on Linux and macOS. The version part comes from `git describe --tags`. See Tagging and releases.

To remove all build artifacts and start over:

```
rm -rf build
```

On Windows `cmd`:

```
rd /s /q build
```

To clean just the object files and module files while keeping the build configuration:

```
cmake --build build --target clean
```

Install

Copy the built executable to a directory of your choice with `cmake --install`. The `--prefix` flag sets the base directory.

```
cmake --install build --prefix ~/bin
```

If you omit `--prefix`, CMake installs to the system default binary location. You can also skip this step and copy the executable yourself.

Linux note on libgfortran

If your Linux distribution does not ship the static `libgfortran.a`, remove the `link_libraries("static")` line from `CMakeLists.txt` and build against the dynamic `libgfortran.so`. The resulting binary then needs the dynamic libraries present at runtime. Use `ldd <binary>` to list them.

If you hit a platform-specific compile failure, open an issue on GitHub with your OS, the error output, and any fix you found.